

Contents

Classification of Floating-Point Numbers.....	1
Introduction	1
Floating point numbers (16 bit).....	1
Types of floating-point number	1
Classification criteria	1
Infinity	1
QNaN	1
SNaN	2
Zero	2
Subnormal	2
Normal	2
Verilog Code	3
Verification	4
Testbench	4
Output.....	6
Conclusion.....	6
Multiplication of floating point	7
Pseudo code	7
Verilog code	9

Classification of Floating-Point Numbers

Introduction

Floating point numbers (16 bit)

The Verilog code is written to take 32, 64 and 128 bit values but for simplicity's sake I shall consider 16 bit floating point representation

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Sign	Exponent					Mantissa									

Types of floating-point number

1. Signalling Not a Number (SNaN)
2. Quiet Not a Number (QNaN)
3. Infinity ($+\infty$, $-\infty$)
4. Zero (-0, +0)
5. Sub-Normal
6. Normal

Classification criteria

Infinity

x	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: = all 1's

Mantissa: = all 0's

Total number of possible values = $2^1 * 2^0 * 2^0 = 2$

QNaN

x	1	1	1	1	1	1	x	x	x	x	x	x	x	x	x
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: all 1's

Mantissa: MSB = 1

Total number of possible values = $2^1 * 2^0 * 2^0 * 2^{10-1} = 2^{10} = 1024$

SNaN

x	1	1	1	1	1	0	x	x	x	x	x	x	x	x	x
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: all 1's

Mantissa: MSB = 0 and at least one 1 (all 0's is not allowed)

Total number of possible values = $2^1 * 2^0 * 2^0 * (2^9 - 1) = 2^{10} - 2 = 1022$

The two missing values are the $+\infty$ and $-\infty$.

Zero

x	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: all 0's

Mantissa: all 0's

Total number of possible values = $2^1 * 2^0 * 2^0 = 2$

Subnormal

x	0	0	0	0	0	x	x	x	x	x	x	x	x	x	x
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: all 0's

Mantissa: all 0's not allowed

Total number of possible values = $2^1 * 2^0 * (2^{10} - 1) = 2^{11} - 2 = 2046$

The two missing values are the $+0$ and -0 .

Normal

x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
Sign	Exponent					Mantissa									

Sign bit: + or –

Exponent: all 0's and all 1's are not allowed

Mantissa: all 0's not allowed

Total number of possible values = $2^1 * (2^5 - 2) * 2^{10} = 2^{16} - 2^{12} = 61,440$

Important flags

1. All bits in exponent are 0's = expZeros
2. All bits in exponent are 1's = expOnes
3. All bits in mantissa are 0's = sigZeros

Verilog Code

```
module hp_class #(
    parameter precision = 16,
    parameter exp_Length = 5,
    parameter exp_index = precision - 1 - exp_Length,
    parameter sign_length = 1
)(
    input [precision - 1 : 0]f,
    output reg SNaN,    // Signalling Not a Number
    output reg QNaN,    // Quite Not a Number
    output reg Infinity,
    output reg Zero,
    output reg SubNormal,
    output reg Normal
);

    wire expZeros; // flag for exponent bit is all zeroes
    wire expOnes;  // flag for exponent bit is all ones
    wire sigZeros; // flag for significand/mantissa is all zeroes

    assign expOnes = & f[precision - 1 - sign_length : exp_index]; // reduction
operator (unary AND)
    assign expZeros = ~| f[precision - 1 - sign_length : exp_index];
    assign sigZeros = ~| f[exp_index-1 : 0]

    assign SNaN      = expOnes & ~sigZeros & ~f[exp_index - 1];
    assign QNaN      = expOnes &  f[exp_index - 1];
    assign Infinity  = expOnes &  sigZeros;
    assign Zero      = expZeros &  sigZeros;
    assign SubNormal = expZeros & ~sigZeros;
    assign Normal    = ~expOnes & ~expZeros;

endmodule
```

Verification

Type of Floating Point	Number of values
SNaN	1022
QNaN	1024
Infinity	2
Zero	2
Subnormal	2046
Normal	61440
Total	65536

We know,

$$2^{16} = 65536$$

Testbench

```
`timescale 1ns / 1ps

module tb_hp_class;
    reg [15:0]f;

    wire SNaN;
    wire QNaN;
    wire Infinity;
    wire Zero;
    wire Subnormal;
    wire Normal;

    integer i, nSNaN, nQNaN, nInfinity, nZero, nSubnormal, nNormal;

    initial begin
        assign f = 16'd0;
        nSNaN = 0;
        nQNaN = 0;
        nInfinity = 0;
        nZero = 0;
        nSubnormal = 0;
        nNormal = 0;
    end

    hp_class uut(f, SNaN, QNaN, Infinity, Zero, Subnormal, Normal);

    initial begin

        for( i = 0; i < 65536; i = i + 1)begin
```

```

#10; assign f = i;

if(SNaN & ~QNaN & ~Infinity & ~Zero & ~Subnormal & ~Normal)begin
    nSNaN = nSNaN + 1;
end
else if(~SNaN & QNaN & ~Infinity & ~Zero & ~Subnormal & ~Normal)begin
    nQNaN = nQNaN + 1;
end
else if(~SNaN & ~QNaN & Infinity & ~Zero & ~Subnormal & ~Normal)begin
    nInfinity = nInfinity + 1;
end
else if(~SNaN & ~QNaN & ~Infinity & Zero & ~Subnormal & ~Normal)begin
    nZero = nZero + 1;
end
else if(~SNaN & ~QNaN & ~Infinity & ~Zero & Subnormal & ~Normal)begin
    nSubnormal = nSubnormal + 1;
end
else if(~SNaN & ~QNaN & ~Infinity & ~Zero & ~Subnormal & Normal)begin
    nNormal = nNormal + 1;
end
else begin
    $display("error");
end
end

begin

$display("Number of SNaN: %d", nSNaN);
$display("Number of QNaN: %d", nQNaN);
$display("Number of Infinity: %d", nInfinity);
$display("Number of Zeroes: %d", nZero);
$display("Number of Subnormal: %d", nSubnormal);
$display("Number of Normal: %d", nNormal);

$display("Total number of values: %d", nSNaN + nQNaN + nInfinity + nZero
+ nSubnormal + nNormal);

#10; $stop;

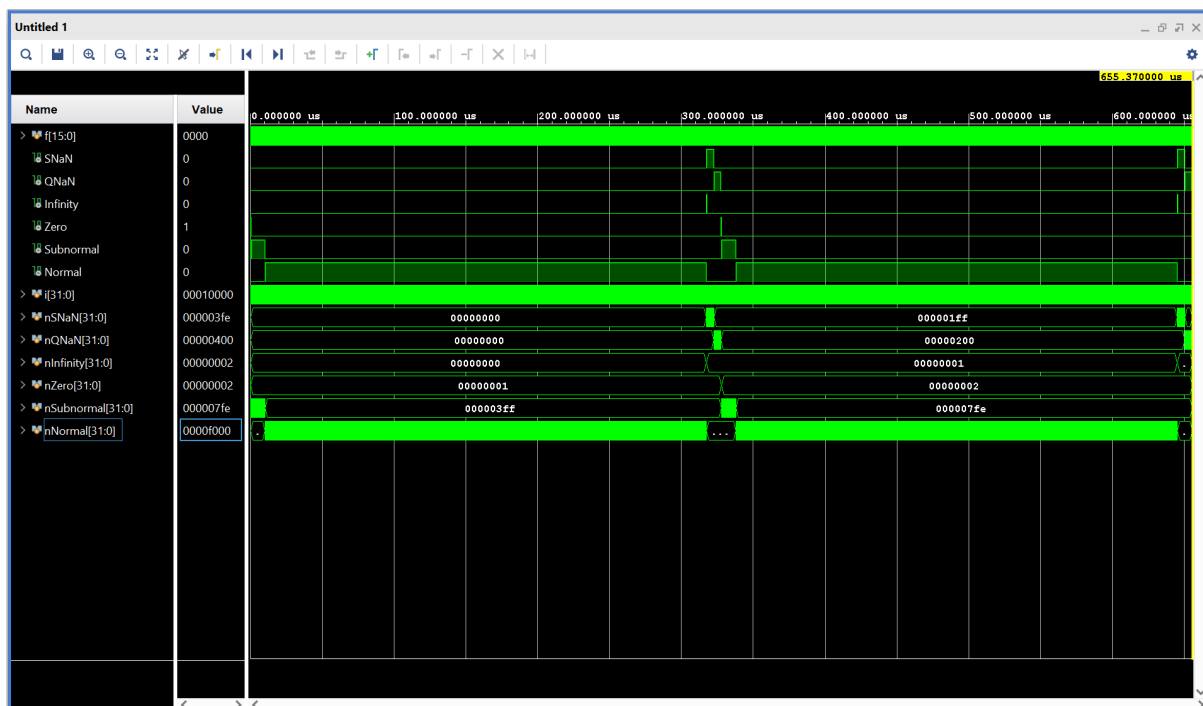
end
end

endmodule

```

Output

```
Number of SNaN:          1022
Number of QNaN:          1024
Number of Infinity:      2
Number of Zeroes:        2
Number of Subnormal:     2046
Number of Normal:        61440
Total number of values:   65536
$stop called at time : 655370 ns : File
"C:/Users/Amogh/Projects/TPS/Floating_Point_Classification_tb_rev_1.
v" Line 67
```



Conclusion

The given FPU classifier accurately classifies the type of floating point as described by the IEEE 754 format and passes one verification test.

Multiplication of floating point

There are 6 classifications of floating point numbers and multiplying two floating point numbers results in $6^2 = 36$ possible combinations of multiplications. By the IEEE 754 specifies how we can deal with some of the cases

	SNaN	QNaN	Infinity	Zero	Subnormal	Normal
SNaN	SNaN	SNaN	SNaN	SNaN	SNaN	SNaN
QNaN	SNaN	QNaN	QNaN	QNaN	QNaN	QNaN
Infinity	SNaN	QNaN	Infinity	QNaN	Infinity	Infinity
Zero	SNaN	QNaN	QNaN	Zero	Zero	Zero
Subnormal	SNaN	QNaN	Infinity	Zero	Zero, Subnormal	Zero, Normal, Subnormal
Normal	SNaN	QNaN	Infinity	Zero	Zero, Normal, Subnormal	Inf, Zero Normal, Subnormal

Pseudo code

Let, a and b be the two floating points being multiplied and p be the product of the multiplication of a and b .

Initialize $p \leftarrow$ canonical NaN pattern

Clear all output flags

$p_{\text{sign}} \leftarrow \text{sign}(a) \oplus \text{sign}(b)$

if a is SNaN or b is SNaN:

$p \leftarrow$ the SNaN operand

set SNaN flag

If a is QNaN or b is QNaN:

$p \leftarrow$ the QNaN operand

Set QNaN flag; return

If a is Inf or b is Inf:

If a is Zero or b is Zero:

Set QNaN flag; $p \leftarrow$ quiet NaN; return

Else:

Set Infinity flag

$p \leftarrow (p_{\text{sign}}, \text{all-ones exponent, zero significand})$

If a is Zero or b is Zero:

Set Zero flag

$p \leftarrow (p_{\text{sign}}, \text{all-zero exponent, zero significand})$

Normalize $a \rightarrow (a_{\text{norm}}, a_{\text{val}})$

Normalize $b \rightarrow (b_{\text{norm}}, b_{\text{val}})$

$p_{\text{exp}} \leftarrow a_{\text{val}} + b_{\text{val}}$

$p_{\text{sig}} \leftarrow (1 \cdot a_{\text{mant}})(1 \cdot b_{\text{mant}})$

If leading bit of p_{sig} is 1: increment p_{exp}

Else if next bit is 1: left shift p_{sig}

If $p_{\text{exp}} > \text{max exponent}$:

Set Infinity flag

$p \leftarrow (p_{\text{sign}}, \text{all-ones exponent, zero significand})$

If $p_{\text{exp}} < \text{minimum subnormal threshold}$:

Set Zero flag

$p \leftarrow (p_{\text{sign}}, \text{all-zero exponent, zero significand})$

If p_{exp} in normal range:

Set Normal flag

$p \leftarrow (p_{\text{sign}}, p_{\text{exp}} + \text{bias, top bits of } p_{\text{sig}})$

If p_{exp} in subnormal range:

Set Subnormal flag

$p \leftarrow (p_{\text{sign}}, 0, p_{\text{sig}} \text{ shifted for subnormal})$

Verilog code

```
module fp_mul #(

////////////////////////////////////////
                change the below parameters only
////////////////////////////////////////

    parameter precision = 16,
    parameter expLength = 5,

////////////////////////////////////////
    parameter signLength = 1,
    parameter sigLength = (precision - signLength - expLength),
    parameter pExpLength = (expLength + 2),
    parameter pSigLength = (2 * (1 + sigLength)),
    parameter flags = 6,
    parameter bias = ((1 << (expLength - 1)) - 1)

)(
    input [precision - 1 : 0]a,
    input [precision - 1 : 0]b,

    output reg [precision - 1 : 0]p,
    output reg pNaN,
    output reg pQNaN,
    output reg pInfinity,
    output reg pZero,
    output reg pSubnormal,
    output reg pNormal
);

reg pSign;

wire aSign, aNaN, aQNaN, aInfinity, aZero, aSubnormal, aNormal;
wire bSign, bNaN, bQNaN, bInfinity, bZero, bSubnormal, bNormal;

wire [precision - 1 : 0]aNew;
wire [precision - 1 : 0]bNew;

reg signed [pExpLength - 1 : 0]pExp;
reg [pSigLength - 1 : 0]pSig;

wire signed [pExpLength - 1 : 0]aVal;
wire signed [pExpLength - 1 : 0]bVal;

fp_class #(.precision(precision), .expLength(expLength)) anum(a, aSign, aNaN,
aQNaN, aInfinity, aZero, aSubnormal, aNormal);
```

```

fp_class #(.precision(precision), .expLength(expLength)) bnum(b, bSign, bSNaN,
bQNaN, bInfinity, bZero, bSubnormal, bNormal);

norm #(.precision(precision), .expLength(expLength)) inst_a1(a, aSubnormal, aNew,
aVal);
norm #(.precision(precision), .expLength(expLength)) inst_b1(b, bSubnormal, bNew,
bVal);

always@(*) begin

    p = {1'b0, {expLength{1'b1}}, 1'b0, {(sigLength - 1){1'b1}}};
    {pSNaN, pQNaN, pInfinity, pZero, pSubnormal, pNormal} = {flags{1'b0}};

    pSign = aSign ^ bSign;

    if (aSNaN | bSNaN) begin
        p = aSNaN == 1'b1 ? a : b;
        pSNaN = 1;
    end

    else if (aQNaN | bQNaN) begin
        p = aQNaN == 1'b1 ? a : b;
        pQNaN = 1;
    end

    else if (aInfinity | bInfinity)begin
        pQNaN = aZero | bZero;
        pInfinity = ~pQNaN;
        p = {pSign, {expLength{1'b1}}, pQNaN, {(sigLength - 1){1'b0}}};
        // add logic for storing '*'
    end

    else if (aZero | bZero) begin
        pZero = 1'b1;
        p = {pSign, {expLength{1'b0}}, {(sigLength){1'b0}}};
    end

    else begin

        pExp = aVal + bVal;
        pSig = ({1'b1, aNew[sigLength - 1 : 0]}) * ({1'b1, bNew[sigLength - 1 :
0]});

        if (pSig[pSigLength - 1]) begin pExp = pExp + 1; end
        else if (~pSig[pSigLength - 1] & pSig[pSigLength - 2]) begin pSig = pSig
<< 1; end

        if (pExp > precision - 1) begin

```

```

        {pSNaN, pQNaN, pInfinity, pZero, pSubnormal, pNormal} = 6'b001000;
        p = {pSign, {expLength{1'b1}}, {sigLength{1'b0}}};
    end

    else if (pExp < -(bias + sigLength - 1)) begin
        {pSNaN, pQNaN, pInfinity, pZero, pSubnormal, pNormal} = 6'b000100;
        p = {pSign, {expLength{1'b0}}, {(sigLength){1'b0}}};
    end

    else if((pExp < precision - 1) & (pExp > -(bias - 1))) begin
        {pSNaN, pQNaN, pInfinity, pZero, pSubnormal, pNormal} = 6'b000001;
        p = {pSign, (pExp[expLength - 1 : 0] + bias), pSig[pSigLength - 2 :
pSigLength - sigLength - 1]};
    end

    else if ((pExp < -(bias - 1)) & (pExp > -(bias + sigLength))) begin
        {pSNaN, pQNaN, pInfinity, pZero, pSubnormal, pNormal} = 6'b000010;
        p = {pSign, {expLength{1'b0}}, (pSig[precision - 1 : precision -
sigLength] >> (-(pExp + bias)))};
    end

    end

end

endmodule

module norm #(

    parameter precision = 16,
    parameter signLength = 1,
    parameter expLength = 5,
    parameter sigLength = (precision - signLength - expLength),
    parameter pExpLength = expLength + 2,
    parameter pSigLength = 2 * (1 + sigLength),
    parameter flags = 6,
    parameter bias = ((1 << (expLength - 1)) - 1)
) (
    input [precision - 1 : 0] x,
    input xSubnormal,

    output reg [precision - 1 : 0] xNew,
    output reg [pExpLength - 1 : 0] xVal
);

integer i, shift;

always @(*) begin

```

```

shift = 0;
if (xSubnormal) begin
    for (i = 0; i < sigLength; i = i + 1) begin
        if (x[i]) begin
            shift = i;
        end
    end
    xVal = x[precision - sigLength - 1 : sigLength] - bias - (sigLength -
shift);
    xNew = {x[precision - 1 : sigLength], (x[sigLength - 1 : 0] << (sigLength -
shift))};
end

else begin
    xVal = x[precision - sigLength - 1 : sigLength] - bias;
    xNew = x;
end
end
endmodule

module fp_class #(
    parameter precision = 16,
    parameter expLength = 5,
    parameter sigLength = 1,
    parameter sigLength = (precision - signLength - expLength)

)(
    input [precision - 1 : 0]f,

    output Sign,
    output SNaN,
    output QNaN,
    output Infinity,
    output Zero,
    output Subnormal,
    output Normal
);

wire expZeros;
wire expOnes;
wire sigZeros;

assign expOnes = & f[precision - 1 - signLength : sigLength]; // reduction
operator (unary AND)
assign expZeros = ~| f[precision - 1 - signLength : sigLength];
assign sigZeros = ~| f[sigLength - 1 : 0];

assign Sign = f[precision - 1];

```

```
assign SNaN      = expOnes & ~sigZeros & ~f[sigLength - 1];
assign QNaN      = expOnes & f[sigLength - 1];
assign Infinity  = expOnes & sigZeros;
assign Zero      = expZeros & sigZeros;
assign Subnormal = expZeros & ~sigZeros;
assign Normal    = ~expOnes & ~expZeros;

endmodule
```

